

CISC 365 - Algorithms I

Lecture 2: Algorithm Complexity

- Attributes of algorithms
- Algorithm efficiency/complexity
- Insertion sort
- Best- vs. worst-case running time
- Order of growth

School of Computing, Queen's University

1

Algorithms in computing

- What computational problems are solved by algorithms?
- What are the different requirements of problems?
 - Only the best solution will do
 - An approximately the best solution is good enough

2

2

Attributes of algorithms

- Any algorithm has, by definition, certain characteristics
- Are some algorithms better than others?
- What are the attributes of algorithms?



3

3

Algorithm correctness

- What is a correct algorithm?
 - If, for **every** input instance, it halts with the correct output.
 - We say that a correct algorithm solves the given computational problem.
- Given two correct algorithms that both solve the same problem, how should we choose between them?
 - Ease of understanding
 - Elegance
 - Efficiency (space and time)

4

4

Algorithm efficiency

- Why should we care about algorithm efficiency?



To sort 10 million numbers



10 billion instructions/second

10 million instructions/second

Insertion Sort $O(n^2)$

Merge Sort $O(n \lg n)$

$(10^7)^2$ instructions

$10^7 \lg 10^7$ instructions

20,000 seconds (more than 5.5 hours)

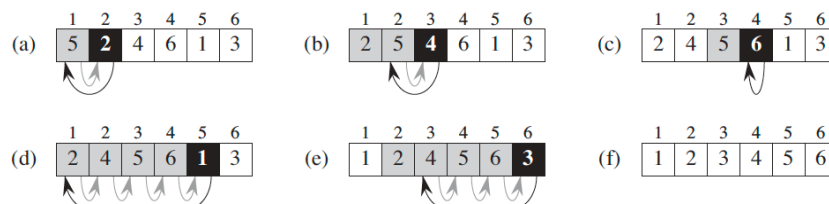
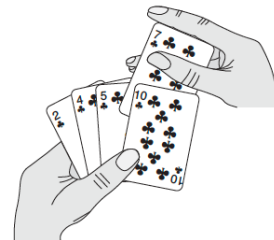
1160 seconds (less than 20 minutes)

5

5

Time complexity of an algorithm

- Insertion Sort



6

6

Time complexity of an algorithm

• Insertion Sort

INSERTION-SORT (<i>A</i>)	<i>cost</i>	<i>times</i>
1 for $j = 2$ to $A.length$	c_1	
2 $key = A[j]$	c_2	
3 // Insert $A[j]$ into the sorted sequence $A[1 \dots j - 1]$.		0
4 $i = j - 1$	c_4	
5 while $i > 0$ and $A[i] > key$	c_5	
6 $A[i + 1] = A[i]$	c_6	
7 $i = i - 1$	c_7	
8 $A[i + 1] = key$	c_8	

*Note that index of list A starts from 1 to A.length

7

7

Complexity of insertion sort

INSERTION-SORT (<i>A</i>)	
1 for $j = 2$ to $A.length$	$T(n) = c_1n + c_2(n-1) + c_4(n-1) + c_5 \sum_{j=2}^n t_j + c_6 \sum_{j=2}^n (t_j - 1)$
2 $key = A[j]$	
3 // Insert $A[j]$ into the sorted sequence $A[1 \dots j - 1]$.	$+ c_7 \sum_{j=2}^n (t_j - 1) + c_8(n-1) .$
4 $i = j - 1$	
5 while $i > 0$ and $A[i] > key$	
6 $A[i + 1] = A[i]$	
7 $i = i - 1$	
8 $A[i + 1] = key$	

• Best case

$$t_j = 1$$

$$\begin{aligned} T(n) &= c_1n + c_2(n-1) + c_4(n-1) + c_5(n-1) + c_8(n-1) \\ &= (c_1 + c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8) . \end{aligned}$$

$$T(n) = an + b$$

8

8

Complexity of insertion sort

INSERTION-SORT(A)

1 **for** $j = 2$ **to** $A.length$

2 $key = A[j]$

3 // Insert $A[j]$ into the sorted
 sequence $A[1..j-1]$.

4 $i = j - 1$

5 **while** $i > 0$ and $A[i] > key$

6 $A[i+1] = A[i]$

7 $i = i - 1$

8 $A[i+1] = key$

$$T(n) = c_1n + c_2(n-1) + c_4(n-1) + c_5 \sum_{j=2}^n t_j + c_6 \sum_{j=2}^n (t_j - 1)$$

$$+ c_7 \sum_{j=2}^n (t_j - 1) + c_8(n-1).$$

• Worst case

$$t_j = j \text{ for } j = 2, 3, \dots, n$$

$$T(n) = c_1n + c_2(n-1) + c_4(n-1) + c_5 \left(\frac{n(n+1)}{2} - 1 \right)$$

$$+ c_6 \left(\frac{n(n-1)}{2} \right) + c_7 \left(\frac{n(n-1)}{2} \right) + c_8(n-1)$$

$$= \left(\frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2} \right) n^2 + \left(c_1 + c_2 + c_4 + \frac{c_5}{2} - \frac{c_6}{2} - \frac{c_7}{2} + c_8 \right) n$$

$$- (c_2 + c_4 + c_5 + c_8).$$

$$T(n) = an^2 + bn + c$$

9

9

Best vs. worst-case running time

• Which one should we consider?

- The worst-case running time of an algorithm gives an upper bound on the running time for any input
- The worst-case occurs fairly often

10

10

Order of growth

n	T1	T2	T3	T4
5	105	70	30	100
10	205	90	110	250
20	405	130	420	850
40	805	210	1640	3250
80	1605	370	6480	12850
160	3205	690		
320	6405	1330		
640	12805	2610		

$$T_1(n) = 20n + 5$$

$$T_1(n) \rightarrow 20n$$

$$T_1(n) \rightarrow n$$

$$T_2(n) = 4n + 50$$

$$T_2(n) \rightarrow 4n$$

$$T_2(n) \rightarrow n$$

$$T_1(n) \text{ is in } \Theta(n)$$

$$T_3(n) = n^2 + n$$

$$T_3(n) \rightarrow n^2$$

$$T_3(n) \rightarrow n^2$$

$$T_3(n) \text{ is in } \Theta(n^2)$$

$$T_4(n) = 2n^2 + 50$$

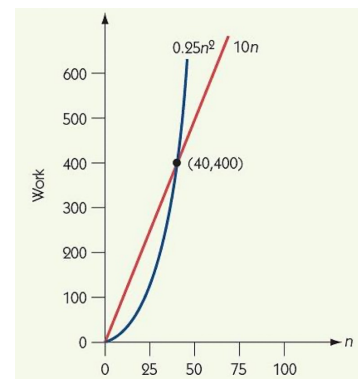
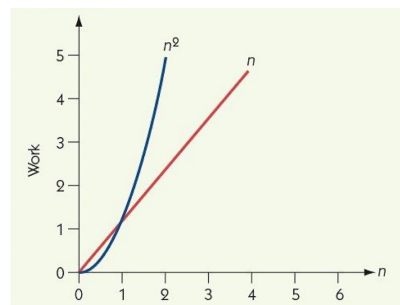
$$T_4(n) \rightarrow 2n^2$$

$$T_4(n) \rightarrow n^2$$

11

11

Order of growth



12

12

Analyzing computational complexity

- For a sequence of statements, consider the complexity of the most complex thing in the sequence
- For a loop, consider the complexity of the body of the loop multiplied by the number of times the loop executes
- For a conditional statement, consider the complexity of whichever of the two alternatives has the higher complexity

13

13

How exactly?

- Identify the fundamental step that is executed the most often
- Write down a function that relates the number of times that step executes to the size of the input
- Simplify that function as much as possible by discarding smaller terms and constant coefficients
- What remains is the complexity of the algorithm/code

14

14

- Insertion Sort

```

INSERTION-SORT(A)
1  for j = 2 to A.length
2    key = A[j]
3    // Insert A[j] into the sorted
      sequence A[1 .. j - 1].
4    i = j - 1
5    while i > 0 and A[i] > key
6      A[i + 1] = A[i]
7      i = i - 1
8    A[i + 1] = key

```

15

15

```

1.
def fubar_1 (a_list):
    n = len(a_list):
    for i in range(n//3):
        a_list[i] *= 2
    for j in range(n//2):
        a_list[j] -= 1
    for k in range(n-1, 0, -1):
        a_list[k] = (a_list[k] + a_list[j] * a_list[i]) % n

```

$$O(n^3)$$

16

16

2.

```
def fubar_2 (a_list):
    n = len(a_list)
    i = 0
    while i + 1 < n:
        a_list[i], a_list[i+1] = a_list[i+1], a_list[i]
        i += 2
    i = 0
    while i + 2 < n:
        a_list[i], a_list[i+1] = a_list[i+1], a_list[i]
        i += 3
```

$$O(n)$$

17

17

3.

```
def fubar_3(a_list):
    n = len(a_list)
    for i in range(n-1):
        j = i + 1
        while j < n:
            if a_list[i] < a_list[j]:
                a_list[i], a_list[j] = a_list[j], a_list[i]
            j += 1
```

$$O(n^2)$$

18

18

4.

```
def fubar_4 (a_list):
    n = len(a_list)
    i = n - 1
    while i >= 0:
        m = n
        j = 0
        while m > 0:
            if a_list[j] < a_list[i]:
                a_list[j], a_list[i] = a_list[i], a_list[j]
            m = m // 2
            j += 1
        i -= 1
```

$O(n \lg n)$

19

19

5.

```
def fubar_5 (a_list):
    n = len(some_list)
    i = n - 1
    while i >= 0:
        m = n
        while m > 0:
            for j in range(n):
                if some_list[j] < some_list[i]:
                    some_list[j], some_list[i] = some_list[i], some_list[j]
            m = m // 2
        i -= 1
```

$O(n^2 \lg n)$

20

20

Reading

- Chapter 2: Getting Started

